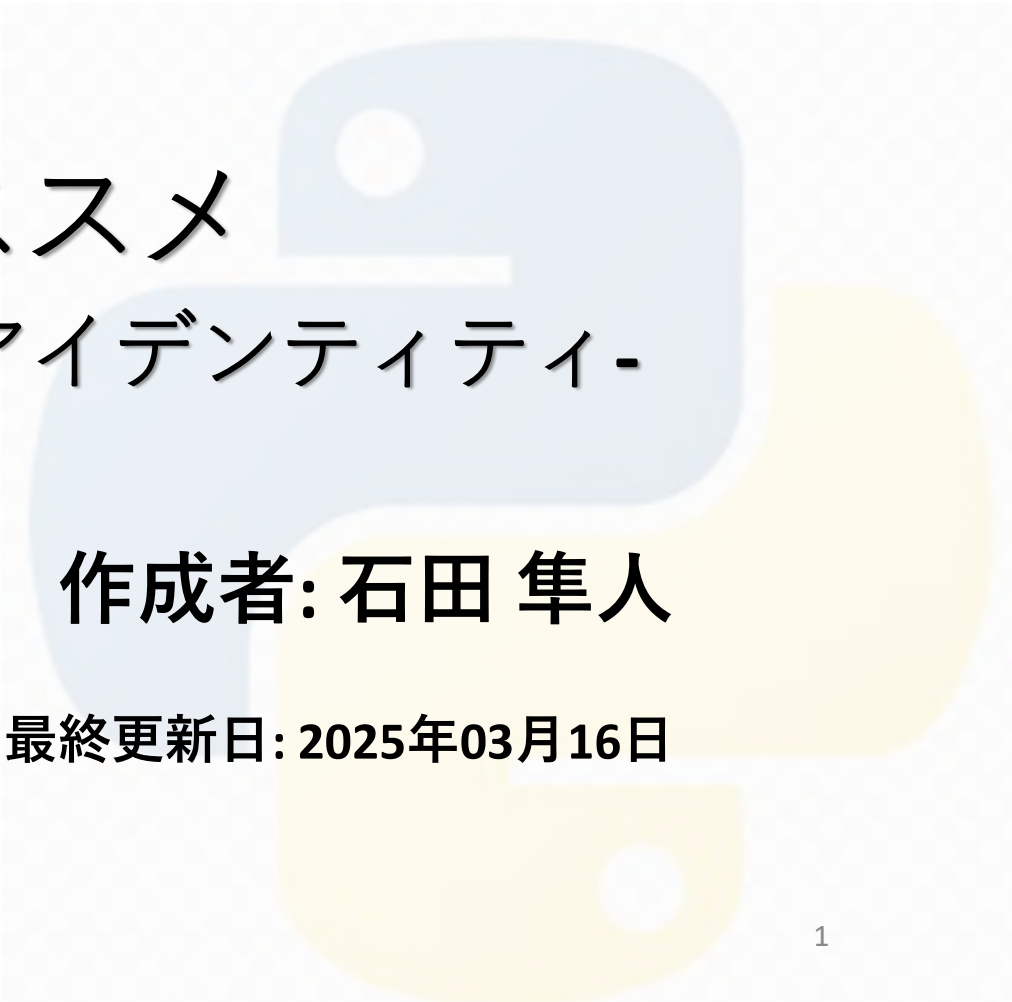
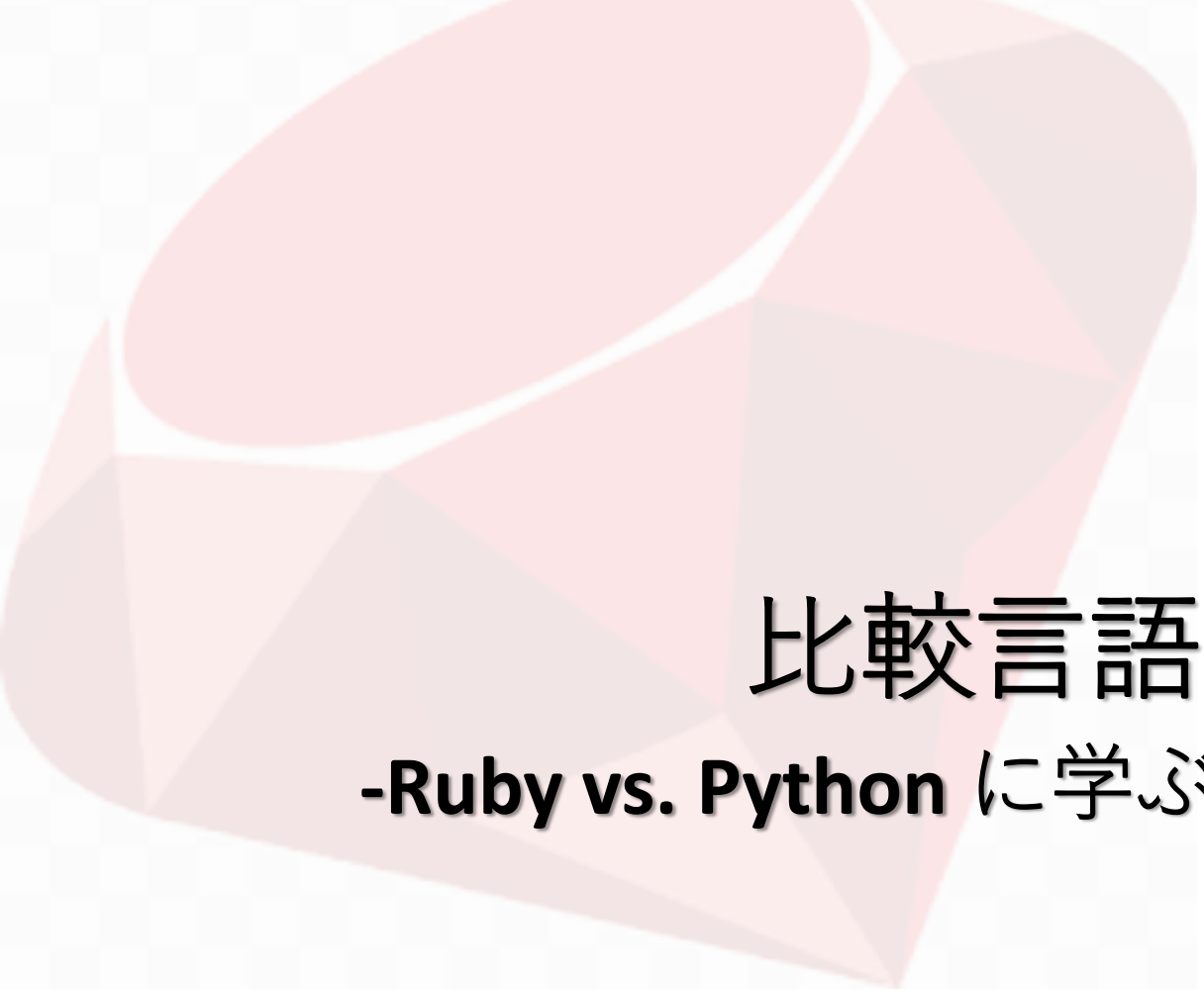




比較言語学のススメ

-Ruby vs. Python に学ぶ言語のアイデンティティ-

作成者: 石田 隼人

英語版は [こちら](#)

最終更新日: 2025年03月16日

自己紹介

- 各種アカウント
 - GitHub: [@hayat01sh1da](#)
 - X: [@hayat01sh1da](#)
 - LinkedIn: [@hayat01sh1da](#)
 - Speaker Deck: [@hayat01sh1da](#)
 - Docswell: [@hayat01sh1da](#)
- 職業: ソフトウェアエンジニア
- 趣味
 - 言語学
 - カラオケ
 - 音楽鑑賞
 - 映画鑑賞
 - 猫飼育



免許 / 資格

- 英語
 - TOEIC® Listening & Reading 915点: 2019年12月 取得
- エンジニアリング
 - 情報セキュリティマネジメント: 2017年11月 取得
 - 応用情報技術者: 2017年06月 取得
 - 基本情報技術者: 2016年11月 取得
 - IT パスポート: 2016年04月 取得
- その他
 - 珠算2級: 2002年06月 取得
 - 暗算3級: 2001年02月 取得

スキルスタック

- 自然言語
 - 日本語: 母語
 - 英語: ビジネスレベル
- 開発
 - **バックエンド**: Ruby(Ruby on Rails), Python
 - フロントエンド: TypeScript + React.js, TypeScript + Vue.js
 - **データベース**: MySQL, PostgreSQL, MongoDB
 - **アーキテクチャー**: モノリス, モジュールモノリス
 - ホスティング: AWS ESK
 - バージョン管理: Git, GitHub
 - **CI/CD**: GitHub Actions, ArgoCD
 - 監視: Datadog, Sentry

職歴

期間	業種	職種	業務内容
2021年08月- Present	EdTech SaaS	ソフトウェア エンジニア	• バックエンド開発 • フロントエンド開発 • CI/CD 運用・保守 • 週次リリースマネージャー • トラブル対応 • ドキュメンテーション • 技術負債解消 • 技術ブログ記事執筆 • LT 登壇
2020年02月- 2020年12月	チャットボット系 SaaS	バックエンド エンジニア	• バックエンド開発 • ドキュメンテーション • 技術負債解消 • 代替 API 検証

職歴

期間	業種	職種	業務内容
2018年06月 - 2020年01月	受託開発	ソフトウェア エンジニア	• バックエンド開発 • フロントエンド開発 • トラブル対応 • QA • 技術ブログ記事執筆
2016年04月 - 2018年01月	SES	システムエンジニア	• 全社アカウント管理者 • Windows Server 運用・保守 • セキュリティ管理者助手 • 翻訳・通訳

国際交流活動

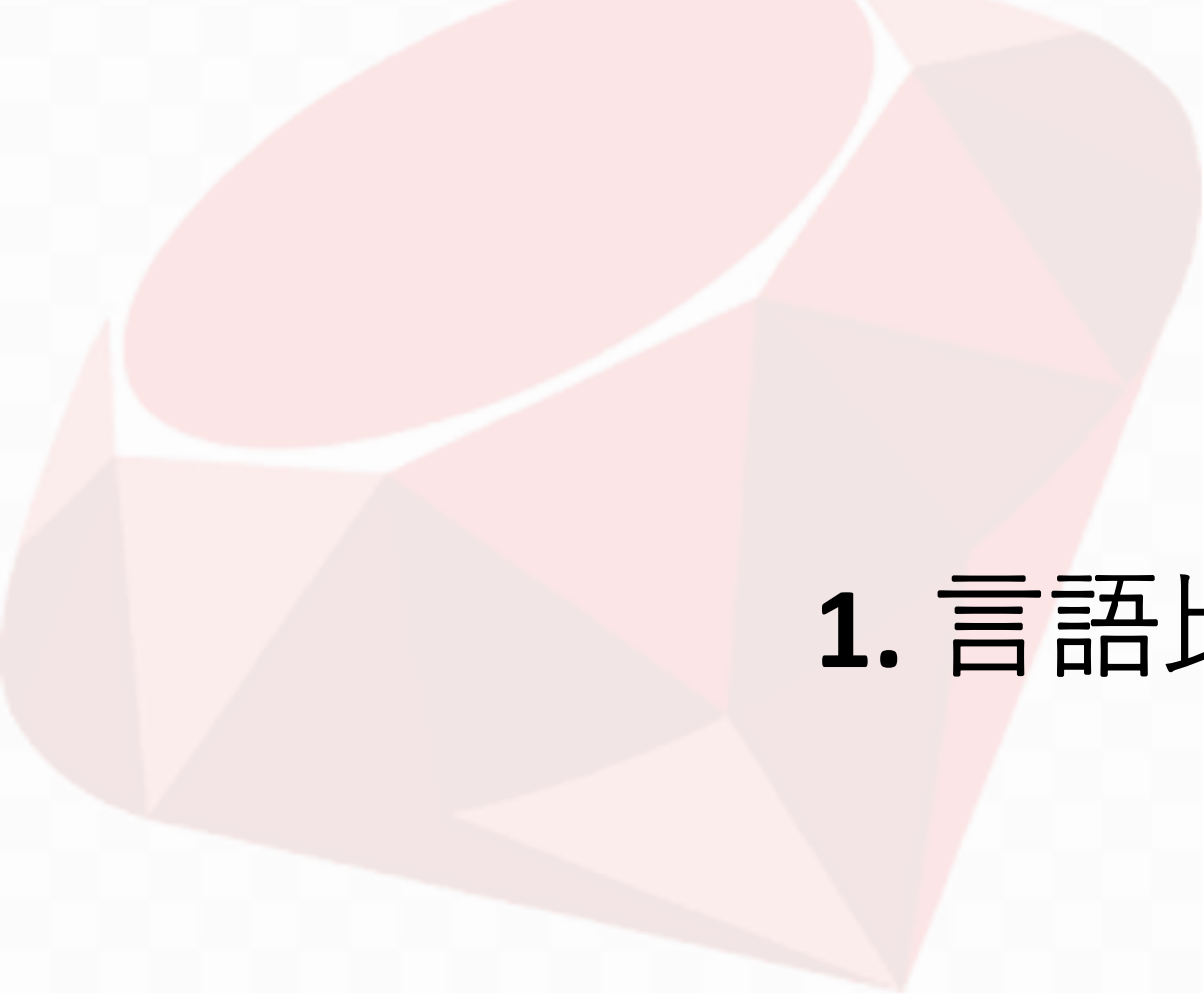
- 大学時代
 - 英語学ゼミ(マスメディア英語)
 - 国際交流サークル兼部(2年次)
 - [内閣府主催国際交流プログラム](#)(2013年 - 2016年)
 - 日本語学の授業の S 単位取得(最終年次)
- 海外生活
 - オーストラリアでのワーキングホリデー(2014年04月 - 2015年03月)
 - 2ヶ月間のシドニーの語学学校通学
 - 6ヶ月間の [Hamilton Island Resort](#) での就労
 - 1ヶ月間の NSW 州の [St Ives High School](#) での日本語教師アシスタントボランティア活動
- その他活動
 - 英語での日々の日記(2014年04月 - 現在)
 - [Sunrise Toastmasters Club](#) 参加(2017年02月 - 2018年03月)
 - [Vital Japan](#) 参加(2018年01月 - 2019年07月、2022年10月 - 2023年02月)
 - 英語自己学習
 - Ruby 関連カンファレンス参加

目次

1. [言語比較の意義](#)
2. [Ruby と Python の三大共通点](#)
3. [Ruby と Python の三大相違点](#)
4. [おわりに](#)
5. [参考文献](#)

VS.





1. 言語比較の意義

1. 言語比較の意義

私は大学生時代は英語学を専攻していた。

そこでの一番の大きな学びは「**物事の本質は比較によって理解出来る**」ということであった。

そのことを表す格言を以下に引用する。

"He who knows no foreign language knows nothing of his mother tongue." by Johann Wolfgang von Goethe

(外国語を1つも知らないものは自身の母語について何も知らないのと等しい。 by ヨハン・ヴォルフガング・フォン・ゲーテ、ドイツ人文豪)

1. 言語比較の意義

自然言語においては、英語をはじめとした外国語を勉強・生活・仕事で使うことによって日本語との違いを嫌でも意識することになるので、自身と母語のアイデンティティについて深く考察することになる。

※ 詳細は拙作「[現代英語を知ろう Vol.5 -ことばの側面-](#)」を参照。

1. 言語比較の意義

プログラミング言語においては、『[プログラマが知るべき97のこと > 26. 言語だけでなく文化も学ぶ](#)』においても Anders Norås 氏が同様のことに言及している。

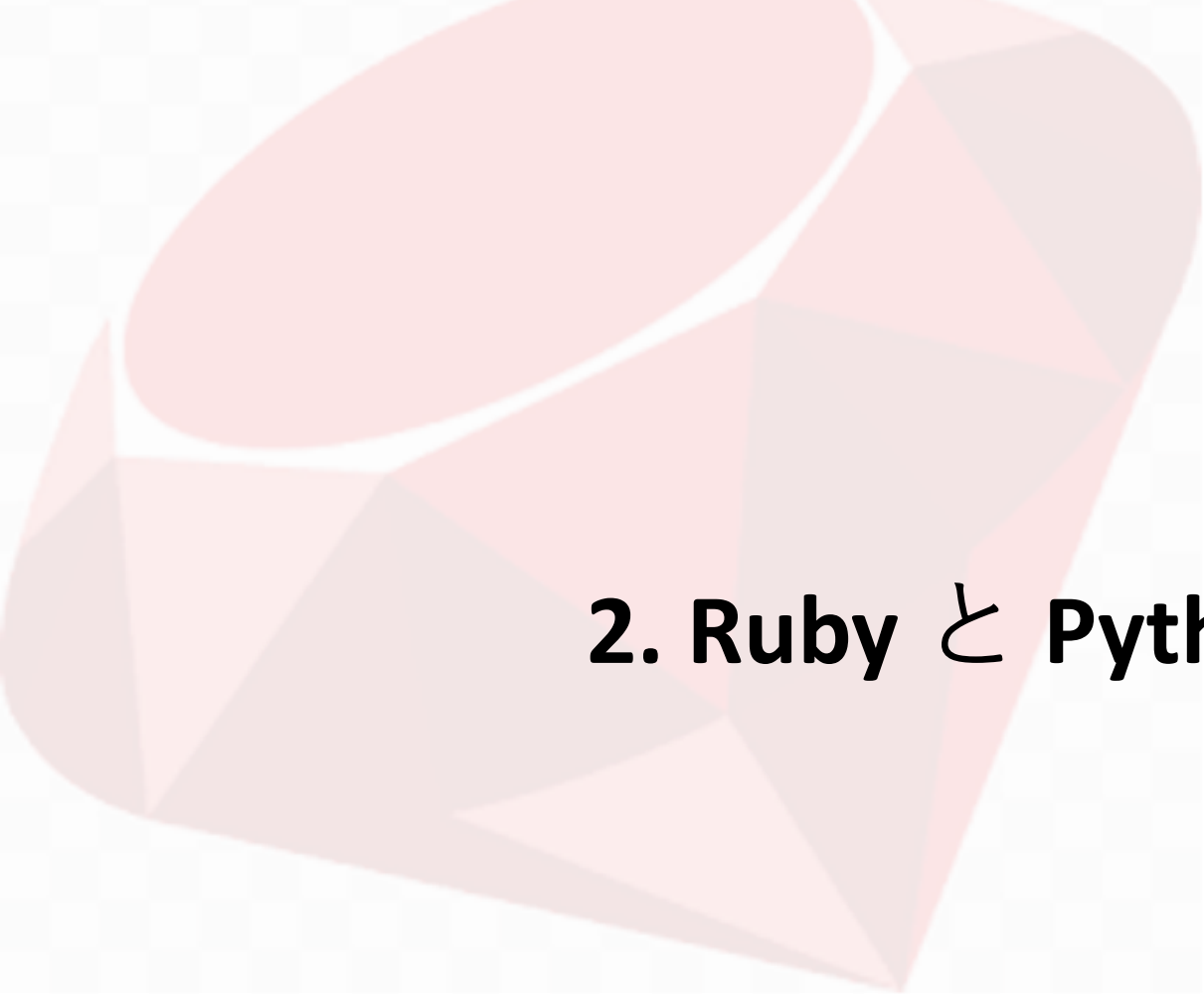
Andy Hunt と Dave Thomas は、多くの人に影特を与えた著書「*速人プログラマ*」の中で、「毎年、新たなプログラミング言語を1つは学ぶこと」と勧めています。私はそのアドバイスに従い、過去何年かの間に実際に数多くの言語を学んできました。そして、その中で「**言語を学ぶというのは、ただ文法、構文を学ぶことではなく、その背景にある文化も学ぶこと**」という重要な教訓を得ました。

1. 言語比較の意義

昨今はソフトウェアエンジニアの責務の広さ・深さともに大きくなっているため、我々が複数言語を扱えることは事実上の必須要件のようになっていと感じている。

折角複数言語を扱うのであれば、**言語比較を通じて第一言語への理解を深める**ことには、スキルセット以上に大きな意義があるはずだ。

LT という特性上時間の制約を大きく受けるため、今回は我々(私とオーディエンスの皆様)の第一言語である **Ruby が何者であるかを意識するきっかけを作る**のがゴールである。



2. Ruby と Python の三大共通点

vs.

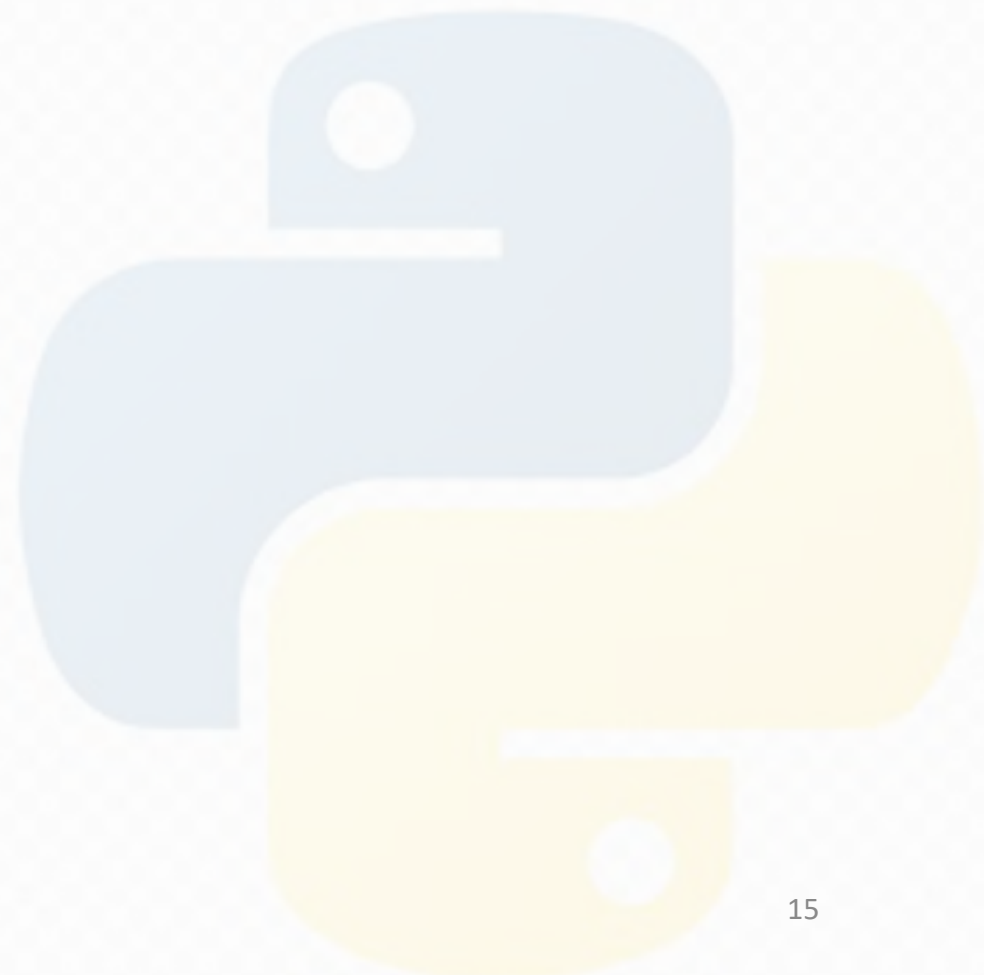


2. Ruby と Python の三大共通点

Ruby と Python には以下3つの大きな共通点がある。

1. オブジェクト指向である
2. 動的型付け言語である
3. 開発者体験重視である

VS.



2. Ruby と Python の三大共通点

① オブジェクト指向である

オブジェクト指向の三原則は以下の通りである。

1. **継承:** 親クラス(基底クラス)の抽象度の高いメソッドや属性を子クラスが継承。子クラスは親クラス(基底クラス)の性質を受け継ぐため、差分のみ実装をすれば良いので再利用性によるコーディング効率化が可能
2. **ポリモーフィズム(多様性):** メソッド呼び出しというメッセージを受け取るレシーバーによって挙動が異なること (e.g., `Doctor#practice` は「～を開業する」という行為を、`Athlete#practice` は「～を練習する」といった具合)
3. **カプセル化:** 内部の詳細な処理やデータを隠蔽し、I/F を通じたレシーバー間のメッセージ(メソッドコールや属性へのアクセス)に起因する外側から見た振舞いのみに興味を持つ仕様のこと。内部処理への強結合を仕様レベルで防ぐことで保守性を担保出来る側面を持つ

2. Ruby と Python の三大共通点

② 動的型付け言語である

コンパイル時にデータ型が決定するのが「静的型付け」、実行時にデータ型が変数のデータによって決定するのが「動的型付け」である。

前者は C ・ Java ・ TypeScript、後者は Ruby ・ Python ・ JavaScript ・ PHP などが当てはまる。

Ruby や Python などの動的型付け言語は実装者である人間が干渉できないため、実行時まで変数にどんな型のデータが入っているかが分からない。

そのため、それが一見して分かるような変数の命名は特に重要である。

『[プログラマが知るべき97のこと > 107. 名前重要](#)』において Ruby の生みの父であるまつもとゆきひろ氏(以降、Matz)が言及しているが、適切な命名は適切な設計の鑑である。

2. Ruby と Python の三大共通点

③ 開発者体験重視である

Ruby や Python は実行が容易である、動的プログラミング(e.g., ``eval`` 関数・``Module#class_eval``)が可能であるなど、その強力な特長を活かした良質なコーディング体験を開発者に提供してくれる。

雑に書いたプログラムですら動いてしまうので、「大いなる力には大いなる責任が伴う」の格言通り、**一定以上の力量を持って駆使しなければ可読性の低下と煩雑なデバッグ体験などのトレードオフに痛いしっぺ返しを食らうことになる。**

要件に基づいて仕様を明らかにし、そこから逆算して必要な実装をクラス・メソッドあたりの責務の範囲を最小限に絞って行う必要がある。ここまで話せばテストコードの重要性は言わずもがな、自明であろう。



3. Ruby と Python の三大相違点

vs.



3. Ruby と Python の三大相違点

Ruby と Python には以下3つの大きな相違点がある。

1. Private フィールドの概念の有無
2. 遅延評価での返り値キャッシュ手法
3. レシーバーと引数

vs.

3. Ruby と Python の三大相違点

① Private フィールドの概念の有無

Ruby には `private` キーワード以下を Private フィールドとして扱う機構が存在する。

一方、Python には Private フィールドの概念そのものがない。

3. Ruby と Python の三大相違点

① Private フィールドの概念の有無

```
1 class User
2   def initialize(name:, age:, occupation:, favourites: [])
3     @name = name
4     @age = age
5     @occupation = occupation
6     @favourites = favourites
7   end
8
9   def greet
10    | "Hi, I'm-#{name}-and-#{age}-years-old.\nMy-occupation-is-#{occupation}.\nMy-favourite-#{inflect('language')}-#{inflect('be')}-#{favourite_programming_language_names}."
11  end
12
13  private
14
15  attr_reader :name, :age, :occupation, :favourites
16
17  def favourite_programming_languages
18    | @favourite_programming_languages ||= favourites.select { |it| it[:genre].equal?(:programming) }
19  end
20
21  def favourite_programming_language_names
22    | return 'nothing' if favourite_programming_languages.empty?
23    | favourite_programming_languages.map { |it| it[:language] }.insert(-2, 'and').join(', ').gsub(/and/, ', ' and ')
24  end
25
26  def inflect(obj)
27    | case obj
28    | when 'be'
29      | (0..1).include?(favourite_programming_languages.length) ? 'is' : 'are'
30    | when 'language'
31      | (0..1).include?(favourite_programming_languages.length) ? obj : "#{obj}s"
32    | end
33  end
34
35  favourites = [
36    | {genre: :programming, language: 'Ruby', level: 'Upper-Intermediate', years_of_experience: 7},
37    | {genre: :programming, language: 'Python', level: 'Intermediate', years_of_experience: 7},
38    | {genre: :programming, language: 'JavaScript', level: 'Intermediate', years_of_experience: 7}
39  ]
40
41  user = User.new(name: 'Hayato-Ishida', age: 33, occupation: 'Software-Engineer', favourites:)
42  puts user.greet
```

```
1 import re
2
3 class User:
4     def __init__(self, name, age, occupation, favourites = []):
5         self.name = name
6         self.age = age
7         self.occupation = occupation
8         self.favourites = favourites
9         self.favourite_programming_languages = self.__favourite_programming_languages__()
10
11     def greet(self):
12         return "Hi, I'm-{name}-and({age})-years-old.\nMy-occupation-is-{occupation}.\nMy-favourite-{language}-{be-verb}-{favourite_programming_language_names}.".format(
13             name = self.name,
14             age = self.age,
15             occupation = self.occupation,
16             language = self.__inflect__('language'),
17             be_verb = self.__inflect__('be'),
18             favourite_programming_language_names = self.__favourite_programming_language_names__()
19         )
20
21     # private
22
23     def __favourite_programming_languages__(self):
24         favourite_programming_languages = []
25
26         for favourite in self.favourites:
27             if favourite['genre'] == 'programming':
28                 favourite_programming_languages.append(favourite)
29
30         return favourite_programming_languages
31
32     def __favourite_programming_language_names__(self):
33         if len(self.favourite_programming_languages) == 0:
34             return 'nothing'
35
36         favourite_programming_language_names = []
37
38         for favourite_programming_language in self.favourite_programming_languages:
39             favourite_programming_language_names.append(favourite_programming_language['language'])
40
41         favourite_programming_language_names.insert(-1, 'and')
42         return re.sub('and', ', ', ', '.join(favourite_programming_language_names))
43
44     def __inflect__(self, obj):
45         match obj:
46             case 'be':
47                 if len(self.favourite_programming_languages) in range(0, 2):
48                     return 'is'
49                 else:
50                     return 'are'
51             case 'language':
52                 if len(self.favourite_programming_languages) in range(0, 2):
53                     return obj
54                 else:
55                     return '{obj}s'.format(obj = obj)
56
57 if __name__ == '__main__':
58     favourites = [
59         | {genre: 'programming', language: 'Ruby', level: 'Upper-Intermediate', years_of_experience: 7},
60         | {genre: 'programming', language: 'Python', level: 'Intermediate', years_of_experience: 7},
61         | {genre: 'programming', language: 'JavaScript', level: 'Intermediate', years_of_experience: 7}
62     ]
63
64     user = User(name = 'Hayato-Ishida', age = 33, occupation = 'Software-Engineer', favourites = favourites)
65     print(user.greet())
```

3. Ruby と Python の三大相違点

① Private フィールドの概念の有無

「[2. Ruby と Python の三大共通点](#) > ① オブジェクト指向である > [3. カプセル化](#)」で言及した**内部の詳細な処理やデータを隠蔽する機構がオブジェクト言語の大きな特徴だが、Python にはこれが存在しない。**

これはクラス外でもデータの書き換えやメソッドコール・オーバーライドが可能なので、バグが混入する可能性が高く危険である。

また、`def \_method` や `def \_\_method\_\_` のように定義して暗黙的にプライベートメソッドとして扱うことは出来るが、機構に保障されていないので安全ではない。

また、ライブラリのメソッドは `\_` で始まる命名が多いため、思わぬオーバーライドを誘発することも Python ユーザーには嘆かれていることは X と Quora で観測している。

3. Ruby と Python の三大相違点

② 遅延評価での返り値キャッシュ手法

言語によって利用可能な記号が異なる。

その1つが`||`である。

Ruby において当該記号は返り値のメモ化(キャッシュ)でよく使われる。

一方、Python では当該記号は使えないため、Ruby のようにメソッド定義で遅延評価でのキャッシュを行うことは出来ない。

Python では、次スライドのようにメソッドの返り値をインスタンス変数に代入することでキャッシュすることしか出来ない模様。

3. Ruby と Python の三大相違点

② 遅延評価での返り値キャッシュ手法

```

1 class User
2   You, 25 minutes ago • Commit sample code in Ruby
3   def initialize(name:, age:, occupation:, favourites: [])
4     @name = name
5     @age = age
6     @occupation = occupation
7     @favourites = favourites
8   end
9
10  def greet
11    "Hi, I'm #{@name} and #{@age} years old. \nMy occupation is #{@occupation}. \nMy favourite: #{inflect('language', @favourites.first)}."
12  end
13
14  private
15
16  attr_reader :name, :age, :occupation, :favourites
17
18  def favourite_programming_languages
19    @favourite_programming_languages ||= favourites.select { |f| f[:genre].equal?('programming') }
20  end
21
22  def favourite_programming_language_names
23    return 'nothing' if favourite_programming_languages.empty?
24    favourite_programming_languages.map { |it| it[:language] }.insert(-2, 'and').join(', ').gsub(/and\./, 'and')
25  end
26
27  def inflect(obj)
28    case obj
29    when 'be'
30      (0..1).include?(favourite_programming_languages.length) ? 'is' : 'are'
31    when 'language'
32      (0..1).include?(favourite_programming_languages.length) ? obj : "#{obj}s"
33    end
34  end
35
36  favourites = [
37    { genre: :programming, language: 'Ruby', level: 'Upper-Intermediate', years_of_experience: 7 },
38    { genre: :programming, language: 'Python', level: 'Intermediate', years_of_experience: 7 },
39    { genre: :programming, language: 'JavaScript', level: 'Intermediate', years_of_experience: 7 }
40  ]
41
42  user = User.new(name: 'Hayato Ishida', age: 33, occupation: 'Software Engineer', favourites: favourites)
43  puts user.greet

```

```

1 import re
2 You, 2 minutes ago | 1 author (You)
3 class User:
4     def __init__(self, name, age, occupation, favourites = []):
5         self.name = name
6         self.age = age
7         self.occupation = occupation
8         self.favourites = favourites
9         self.favourite_programming_languages = self._favourite_programming_languages_()
10
11     def greet(self):
12         return "Hi, I'm {name}-and {age}-years-old.\nMy occupation is {occupation}.\nMy favourite {language} {be_verb} {favourite_programming_language_names}.".format(
13             name = self.name,
14             age = self.age,
15             occupation = self.occupation,
16             language = self._infect__('language'),
17             be_verb = self._infect__('be'),
18             favourite_programming_language_names = self._favourite_programming_language_names_()
19         )
20
21     @private
22     def _favourite_programming_languages_(self):
23         self.favourite_programming_languages = []
24
25         for favourite in self.favourites:
26             if favourite['genre'] == 'programming':
27                 self.favourite_programming_languages.append(favourite)
28
29         return self.favourite_programming_languages
30
31     def _favourite_programming_language_names_(self):
32         if len(self.favourite_programming_languages) == 0:
33             return 'nothing'
34
35         self.favourite_programming_language_names = []
36
37         for favourite_programming_language in self.favourite_programming_languages:
38             self.favourite_programming_language_names.append(favourite_programming_language['language'])
39
40         self.favourite_programming_language_names.insert(-1, 'and')
41
42         return re.sub(r'and,', ' and ', '.').join(self.favourite_programming_language_names)
43
44     def _infect__(self, obj):
45         match obj:
46             case 'be':
47                 if len(self.favourite_programming_languages) in range(0, 2):
48                     return 'is'
49                 else:
50                     return 'are'
51             case 'language':
52                 if len(self.favourite_programming_languages) in range(0, 2):
53                     return obj
54                 else:
55                     return '{obj}s'.format(obj = obj)
56
57 if __name__ == '__main__':
58     favourites = [
59         {'genre': 'programming', 'language': 'Ruby', 'level': 'Upper-Intermediate', 'years_of_experience': 7},
60         {'genre': 'programming', 'language': 'Python', 'level': 'Intermediate', 'years_of_experience': 7},
61         {'genre': 'programming', 'language': 'JavaScript', 'level': 'Intermediate', 'years_of_experience': 7}
62     ]
63
64     user = User(name = 'Hayato Ichida', age = 33, occupation = 'Software Engineer', favourites = favourites)
65     print(user.greet())

```

3. Ruby と Python の三大相違点

③ レシーバーと引数

Ruby と Python を交互に書いていて、同じ用途・同じ名前の組み込みメソッドをコールする際にレシーバーと引数を逆に書いてしまうことがよくある。その一例が Ruby の `Array#join`、Python の `String#join` だ。

- `Array#join` in Ruby: 結合対象の配列をレシーバーに、区切り文字を引数に取る
- `String#join` in Python: 区切り文字をレシーバーに、結合対象の配列を引数に取る

3. Ruby と Python の三大相違点

③ レシーバーと引数

```
1 class User
2   def initialize(name:, age:, occupation:, favourites: [])
3     @name = name
4     @age = age
5     @occupation = occupation
6     @favourites = favourites
7   end
8
9   def greet
10    "Hi, I'm-#{name}-and-#{age}-years-old.\nMy-occupation-is-#{occupation}.\nMy-favourite-#{inflect('language')}-#{inflect('be')}-#{favourite_programming_language_names}."
11  end
12
13  private
14
15  attr_reader :name, :age, :occupation, :favourites
16
17  def favourite_programming_languages
18    @favourite_programming_languages ||= favourites.select { |it| it[:genre].equal?(:programming) }
19  end
20
21  def favourite_programming_language_names
22    return 'nothing' if favourite_programming_languages.empty?
23    favourite_programming_languages.map { |it| it[:language] }.insert(-2, 'and').join(', ').'gsub(/and\,/, 'and')
24  end
25
26  def inflect(obj)
27    case obj
28    when 'be'
29      (0..1).include?(favourite_programming_languages.length) ? 'is' : 'are'
30    when 'language'
31      (0..1).include?(favourite_programming_languages.length) ? obj : "#{obj}s"
32    end
33  end
34
35  favourites = [
36    { :genre => :programming, :language => 'Ruby', :level => 'Upper-Intermediate', :years_of_experience => 7 },
37    { :genre => :programming, :language => 'Python', :level => 'Intermediate', :years_of_experience => 7 },
38    { :genre => :programming, :language => 'JavaScript', :level => 'Intermediate', :years_of_experience => 7 }
39  ]
40
41  user = User.new(name: 'Hayato-Ishida', age: 33, occupation: 'Software-Engineer', favourites:)
42  puts user.greet
```

```
1 import re
2
3 class User:
4     def __init__(self, name, age, occupation, favourites = []):
5         self.name = name
6         self.age = age
7         self.occupation = occupation
8         self.favourites = favourites
9         self.favourite_programming_languages = self.__favourite_programming_languages__()
10
11     def greet(self):
12         return "Hi, I'm-#{name}-and-#{age}-years-old.\nMy-occupation-is-#{occupation}.\nMy-favourite-#{language}-#{be_verb}-#{favourite_programming_language_names}.".format(
13             name = self.name,
14             age = self.age,
15             occupation = self.occupation,
16             language = self.__inflect__('language'),
17             be_verb = self.__inflect__('be'),
18             favourite_programming_language_names = self.__favourite_programming_language_names__()
19         )
20
21     # private
22
23     def __favourite_programming_languages__(self):
24         favourite_programming_languages = []
25
26         for favourite in self.favourites:
27             if favourite['genre'] == 'programming':
28                 favourite_programming_languages.append(favourite)
29
30         return favourite_programming_languages
31
32     def __favourite_programming_language_names__(self):
33         if len(self.favourite_programming_languages) == 0:
34             return 'nothing'
35
36         favourite_programming_language_names = []
37
38         for favourite_programming_language in self.favourite_programming_languages:
39             favourite_programming_language_names.append(favourite_programming_language['language'])
40
41         favourite_programming_language_names.insert(-2, 'and')
42         return re.sub(r'and,', 'and, ', ' '.join(favourite_programming_language_names))
43
44     def __inflect__(self, obj):
45         match obj:
46             case 'be':
47                 if len(self.favourite_programming_languages) in range(0, 2):
48                     return 'is'
49                 else:
50                     return 'are'
51             case 'language':
52                 if len(self.favourite_programming_languages) in range(0, 2):
53                     return obj
54                 else:
55                     return '{obj}s'.format(obj = obj)
56
57 if __name__ == '__main__':
58     favourites = [
59         { 'genre': 'programming', 'language': 'Ruby', 'level': 'Upper-Intermediate', 'years_of_experience': 7 },
60         { 'genre': 'programming', 'language': 'Python', 'level': 'Intermediate', 'years_of_experience': 7 },
61         { 'genre': 'programming', 'language': 'JavaScript', 'level': 'Intermediate', 'years_of_experience': 7 }
62     ]
63
64     user = User(name = 'Hayato-Ishida', age = 33, occupation = 'Software-Engineer', favourites = favourites)
65     print(user.greet())
```

3. Ruby と Python の三大相違点

③ レシーバーと引数

英語として解釈すると以下のようなになる。

- ``Array#join`` in Ruby: The target elements join together with the delimiter you provide.
- ``String#join`` in Python: The delimiter you provide joins the target element together.

英語においては、情報の発信者⇔受信者間で既出の情報である旧情報(Given Information)を出来るだけ前に、新出の新情報(New Information)を出来るだけ後に置く文末焦点の法則(End-Focus)というものが存在する。
つまり、強調したい情報は文末に置かれる。

3. Ruby と Python の三大相違点

③ レシーバーと引数

その意味で、配列を結合して文字列を生成するにおいて、**Ruby は delimiter(区切り文字)に、Python は結合対象の配列の要素**にそれぞれ関心を持っていることになる。

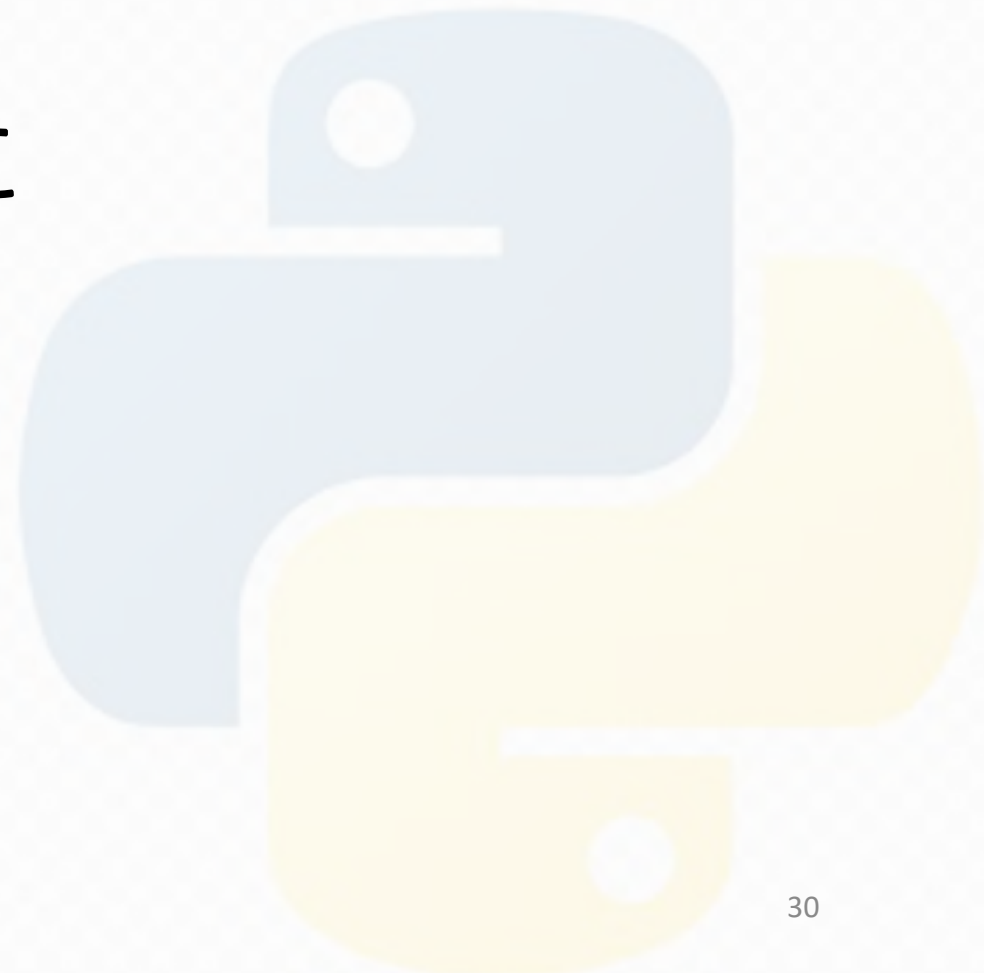
開発者体験の観点で、結合対象の配列の要素を最初に、区切り文字を最後に認知する方が「何が何で結合されるか」を理解しやすくて好ましい。

自然言語同様、第二言語や外国語の習得難易度は母語(第一言語)に依存するため、個人的な感覚かも知れない。

しかし、**Rubyの方が英語を専門にしていた人間の感覚からしても自然言語により近い「書き味」と「可読性」を提供しているのだ。**

5. おわりに

vs.



5. おわりに

5分間という限られた時間の中ではあったが、Python という比較対象を明らかにすることで、普段開発で使っている Ruby がどのような特徴や性質を持つプログラミング言語かを各自で考えるきっかけになっただろうか。

『[プログラマが知るべき97のこと > 43. プログラミング言語は複数習得すべき](#)』の中ではパラダイム(仕組み・枠組み)が異なる言語を習得することが推奨されている。

自然言語でいえば、アルタイ語派の日本語を話す我々がゲルマン語派の英語を習得するようなものである(「?」な方は拙作[「現代英語を知ろう Vol.1 -英語という言語- > 2. 言語と語族と習得難易度の関係」](#)を参照)。

両者の文法・語彙・コロケーションが全く異なるので習得難易度は高い。同じことがプログラミング言語でも言えるということだ。

5. おわりに

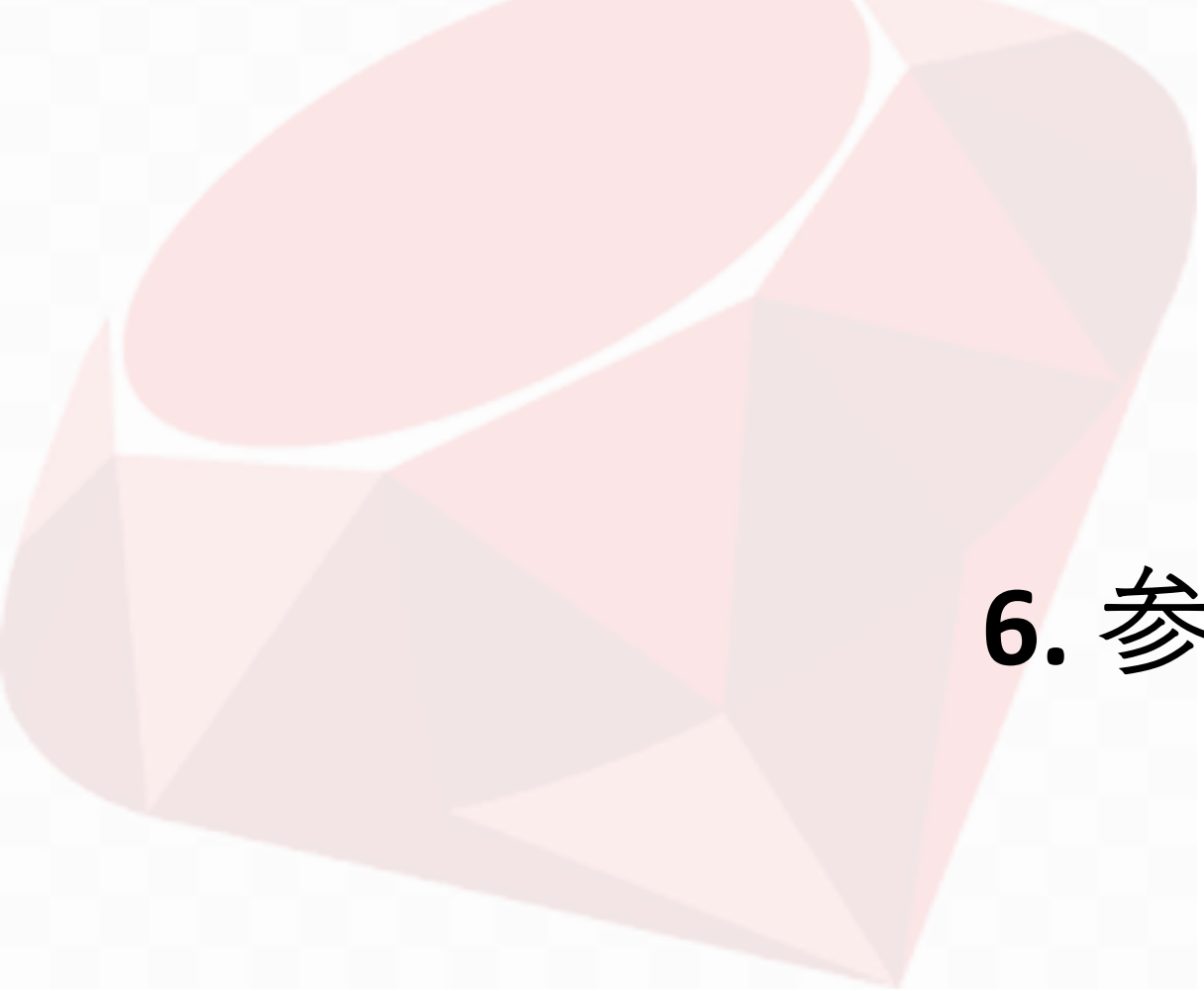
(言語であるかは怪しいが) GitHub Actions でワークフローを組む際に Shell Script を書くことがある。

いつも書いている Ruby の感覚でオブジェクト指向の要領で書こうとすると大抵無理が来る。

配列などは多用せず、単純な処理を共通化する程度の実装が推奨されている。これもパラダイムの違いによるものだ。

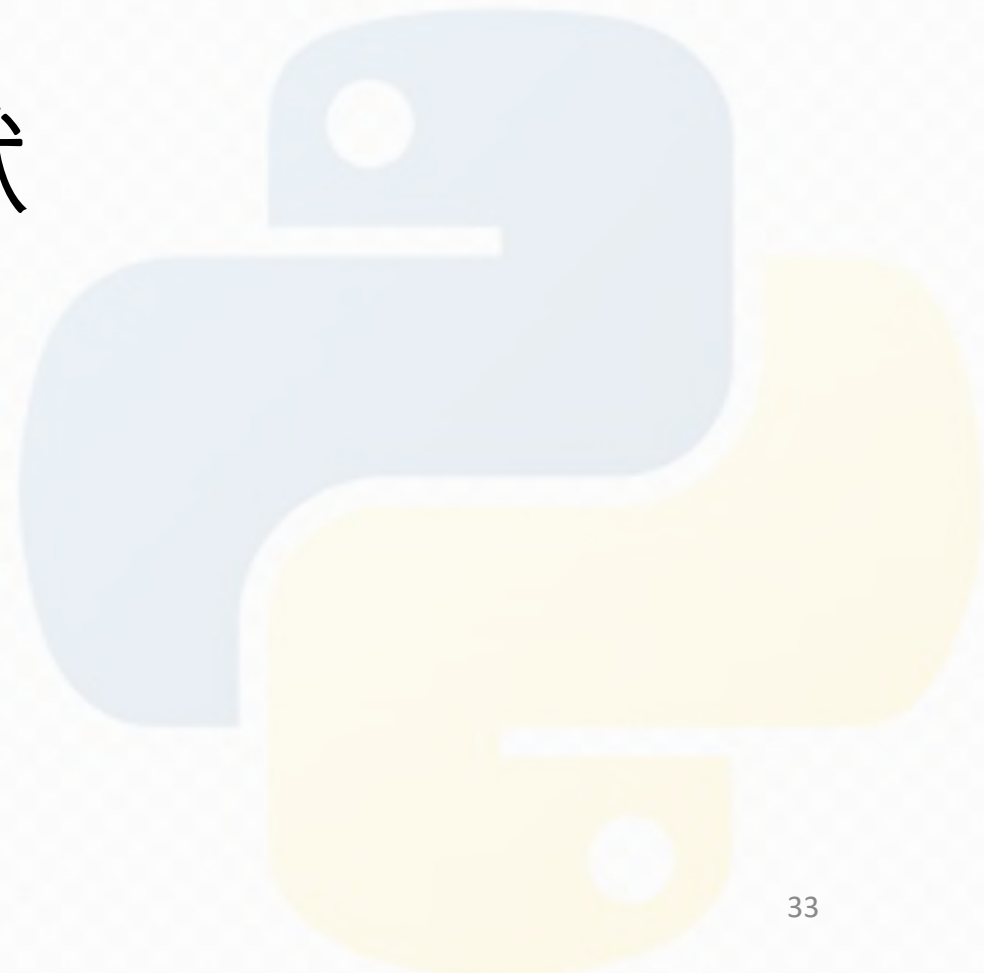
Go や Rust など、データ指向言語は扱ったことがないが、こういった言語を扱う機会があれば同じテーマでまた発表したいと考えている。

すでに習得済みの方がいれば、比較言語学の成果を共有して頂けるととても参考になるので嬉しく思う。



6. 参考文献

vs.



6. 参考文献

- [プログラマが知るべき97のこと](#)
 - 最終アクセス日: 2025年02月10日
- [He who knows no foreign language knows nothing of his mother tongue - La Academia Language School](#)
 - 最終アクセス日: 2025年02月10日
- [オブジェクト指向とは？ 基礎から用語の意味まで分かりやすく解説 - エンジニア type](#)
 - 最終アクセス日: 2025年02月10日
- [コンパイラ言語とインタプリタ言語の違いとは？各言語のメリットも解説 - RENTEQ BLOG](#)
 - 最終アクセス日: 2025年02月10日



EOD
vs.

